ragcap: a multi-agent RAG system, built and measured

A colleague's capstone design (LangGraph router, document RAG with grade and rewrite, SQL agent, general agent, Postgres + pgvector) reproduced on local models, then evaluated the way it should be: frozen test sets, repeated runs, and a version-aware filter with an OFF/ON proof.

Apple M5 Pro · local models · no API key · no admin rights · October 2026

https://github.com/shreenathacc22/ragcap · https://shreenathacc22.github.io/ragcap/

The design to reproduce, and what was added

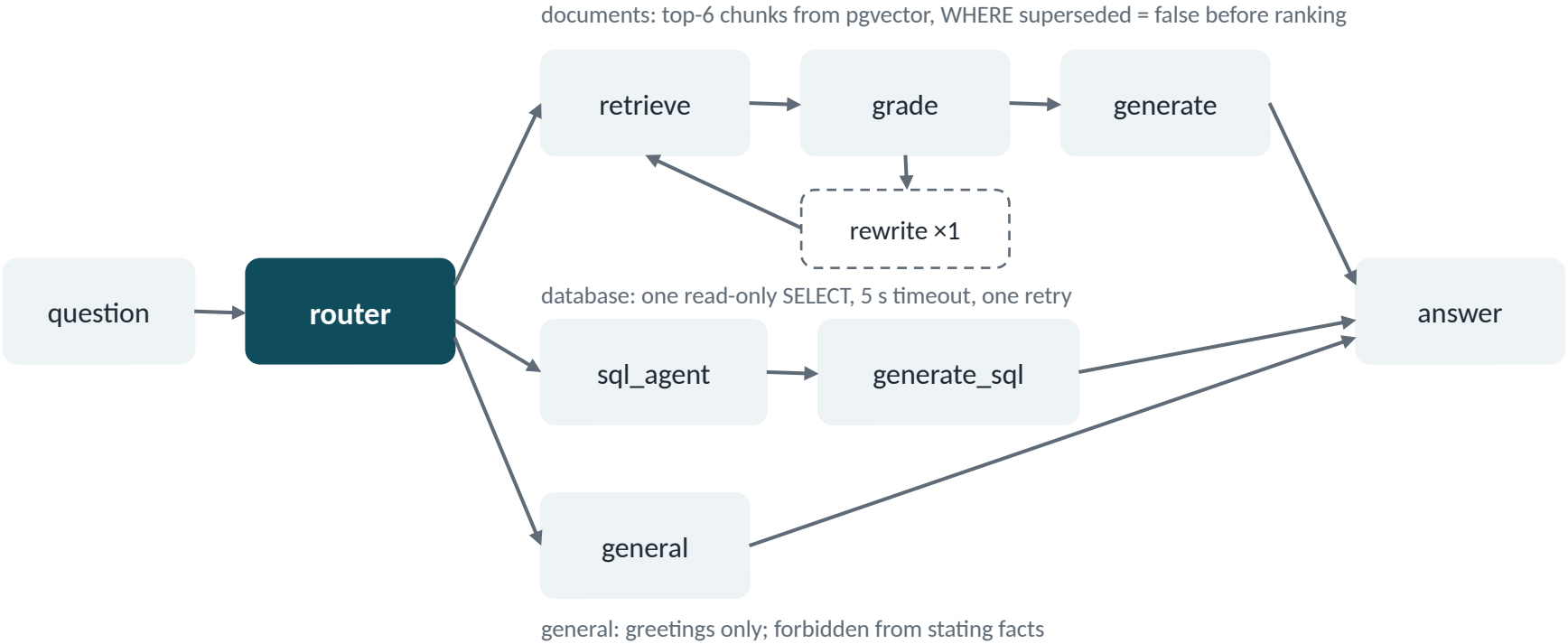
The design (from the capstone PDF)

- Router → documents / database / general
- Retrieve top-6 → Grade → one Rewrite → Generate with citations
- SQL agent: one SELECT, read-only, time-limited, one retry
- General agent never states facts
- Postgres + pgvector, LangGraph, FastAPI, streaming chat, UI
- Reported: latency and cost per flow (110 answers)

Added here

- A quality column: frozen golden set (27), unseen set (25), router / retrieval / answer accuracy
- Repeated runs: means and ranges, failing ids listed
- Version-aware retrieval: superseded versions excluded before ranking, proven OFF vs ON
- Model choice measured: a second-opinion grader lifts unseen answers 80% → 96%
- A P&ID branch on synthetic drawings with a 3B vision model
- Four documents (SOP, runbook, recreate, install); rebuild and clean-start checks

The graph: one router, three paths, one trace



Every node appends its milliseconds and tokens to the trace; the API streams one event per node.

The stack, all local

Postgres 16 + pgvector

Native via pgctl.sh, 127.0.0.1:5433. Tables documents (name, version, superseded, effective_date), chunks (vector(768), HNSW), uploaded_tables.

Ollama

llama3.2:3b for all LLM calls; qwen3:8b as optional grader; nomic-embed-text for embeddings; qwen2.5vl:3b for drawings.

LangGraph 1.2

StateGraph with conditional edges; nodes are plain functions with a timing decorator.

Ingestion

Markdown and PDF to paragraph-packed chunks; front-matter becomes version metadata and is stripped before embedding; CSV/Excel become SQL tables.

FastAPI + web/

/upload, /chat (server-sent events), /eval, /health; a static page renders the node stream. Gradio as a second UI.

Evaluation and operations

eval.py, b1_eval.py (filter OFF/ON), b1_check.py, scenarios.py, pid_eval.py, recreate.sh, services.sh check.

From pre-work to troubleshooting



Pre-work

requirements, design decisions, rules that keep results honest (SOP 3)

Prepare

uv, Python 3.12, Ollama, two required models; no admin rights (SOP 4)

Implement

native Postgres, schema, ingestion, graph, interfaces, P&ID, Docling (SOP 5)

Test

clean start, fingerprints, filter OFF/ON, 18 scenarios, 3 runs per set (SOP 6)

Operate

services.sh, backups, model profiles, change control (runbook)

Troubleshoot

symptom, cause and fix for every problem met (SOP 8, runbook 5)

Every step in the SOP has its commands, the expected output and the result measured on the M5 Pro.

How the numbers are kept honest

1

Freeze first

The golden set was committed before the pipeline existed; the unseen set before its first run. Git order is the evidence.

2

Never tune on the test

A first router draft copied golden questions as examples; it was replaced before any run. Scorer fixes go in an amendments file, never in the frozen set.

3

Commit before measuring

One prompt change at a time, full run, compared by failing case ids, not just the percentage.

4

Repeat and compare

Three runs on each set (identical on this machine); the tuned (92.6%) and unseen (80%) sets are reported side by side.

5

Fair latency

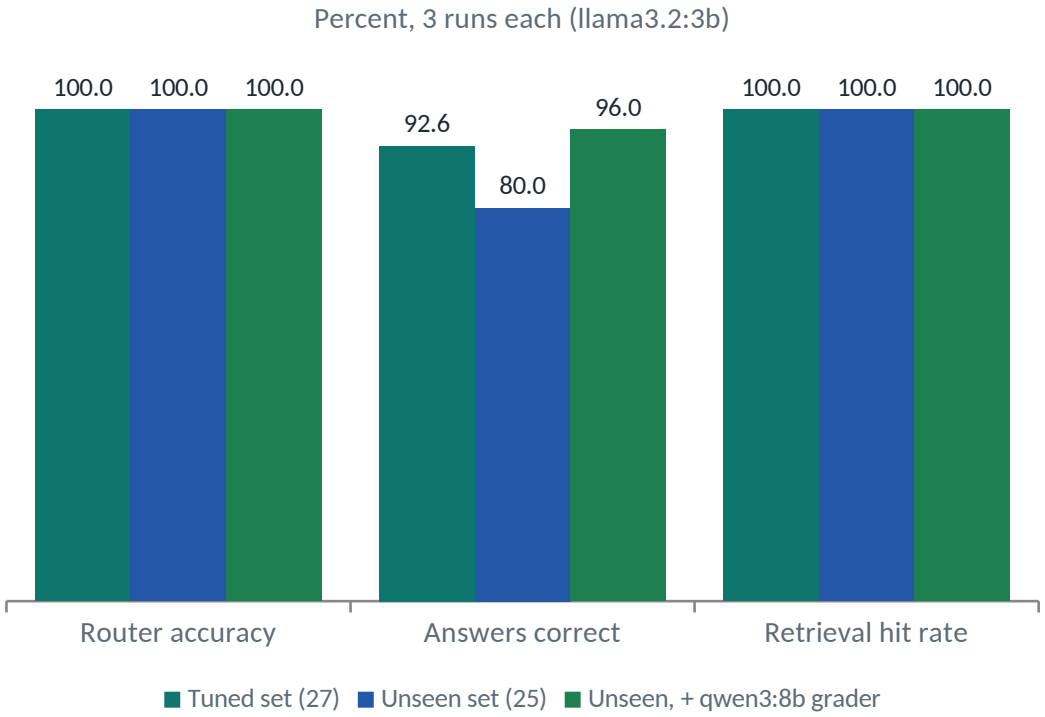
OFF and ON interleaved in one run after a warm-up; quoted as a range (12–13 ms) because runs differ by a millisecond.

6

Score as frozen

llama3.1:8b refused one question in other words; it stays scored as a failure instead of changing the scorer.

Results: tuned set vs unseen set



80 → 96%

unseen answers with a qwen3:8b second-opinion grader that loads only for hard questions

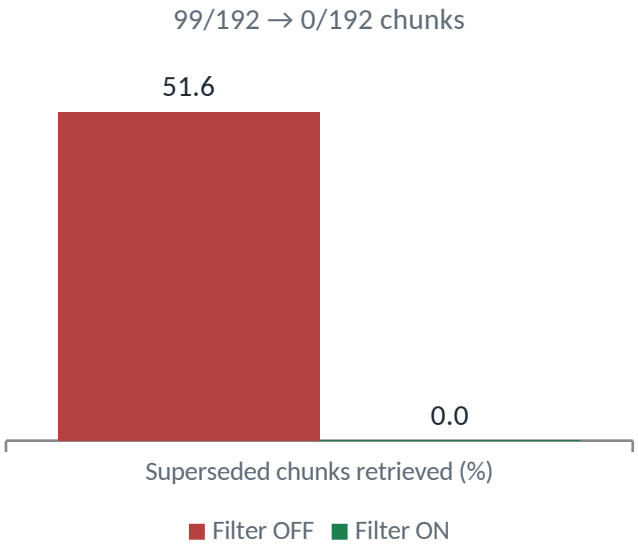
5 cases

documented weaknesses in the scenario catalogue: grader, multi-hop, SQL, two P&ID relations

100%

unanswerable questions refused, every run; general questions never answered with facts

Version-aware retrieval: the trap and the fix



Retrieval only, 32 frozen cases	OFF	ON
Version-trap cases passing (facts present, no stale value from an old chunk)	2/12	12/12
Control cases passing (must stay flat)	20/20	20/20
Median retrieve, interleaved ×3, 96 calls each	12–13 ms	12–13 ms
Pair similarity old vs current (nomic-embed-text)	0.963 – 0.990	

How: each document version carries version, effective_date and superseded; retrieve() adds WHERE d.superseded = false before ORDER BY, so an old runbook that is 0.99-similar to the question is never a candidate. One line of SQL, behind a switch, default ON since the ablation was recorded.

At answer level the filter raised the 27-question score from 88.9% to 92.6% (multi-hop 0/2 → 1/2); version questions were 8/8 both ways because each current runbook names its former value.

Decisions taken on evidence

Second-opinion grader: recommended

qwen3:8b reads the six passages only when the 3B grader keeps none: unseen answers 80% → 96%, about 3 s for a hard question, unloaded after 5 minutes idle.

Next.js: descoped

Node 24 is installed, so it was possible; it adds a toolchain and hosting without changing anything measured. A static page proves the streaming contract instead.

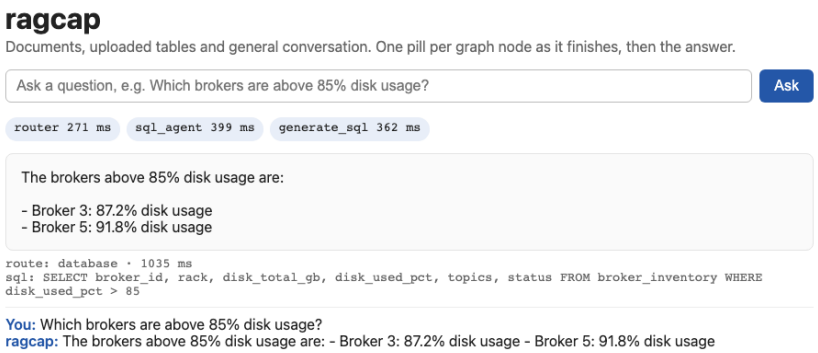
Bigger models: tested, not kept

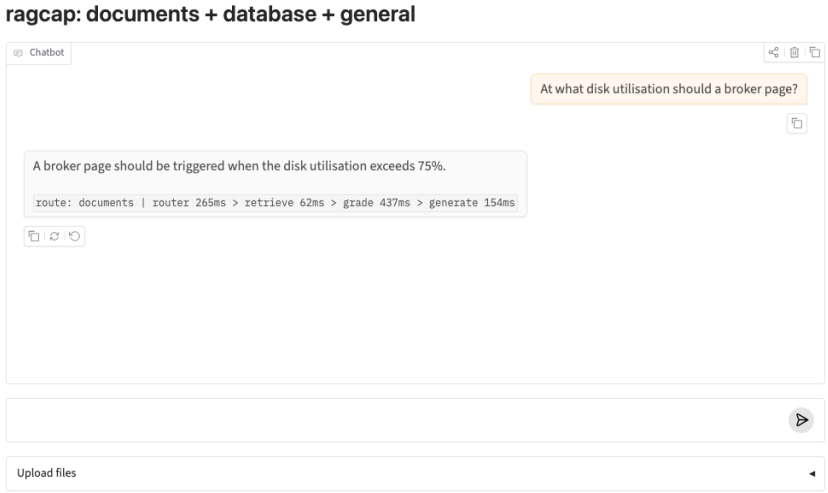
llama3.1:8b for every step scored 92% unseen and stays 5.2 GB resident; qwen2.5vl:7b gave +2 P&ID answers for +2.7 GB. Both were removed.

No Docker, no admin rights

Postgres + pgvector run natively from a pip wheel through pgctl.sh; Ollama is a standalone binary; services.sh check proves a clean start.

Two interfaces on one streaming API

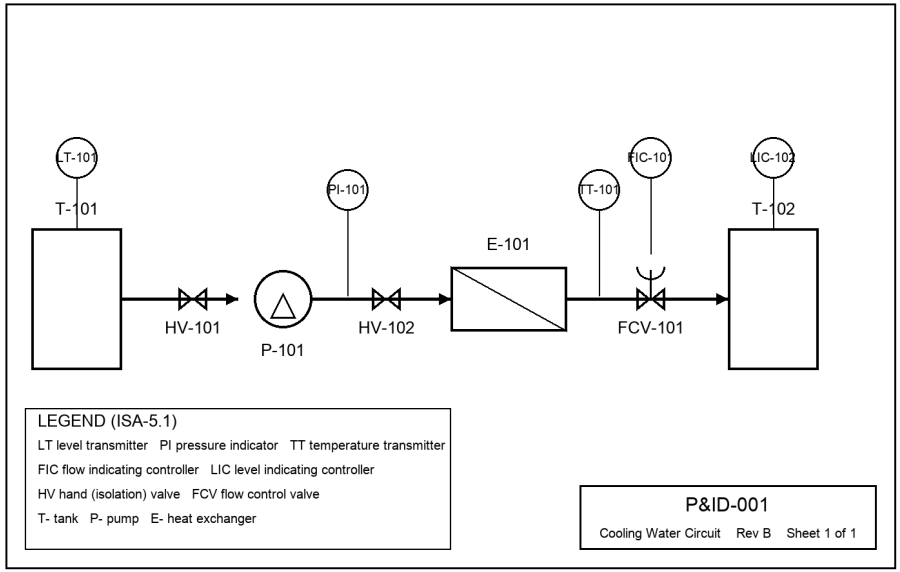




web/index.html: one pill per graph node as it finishes, then the answer, route, SQL and citations. No build step; served by FastAPI at /.

Gradio: chat with the trace line under each answer; upload documents and tables. Both were exercised in headless Chromium with live questions.

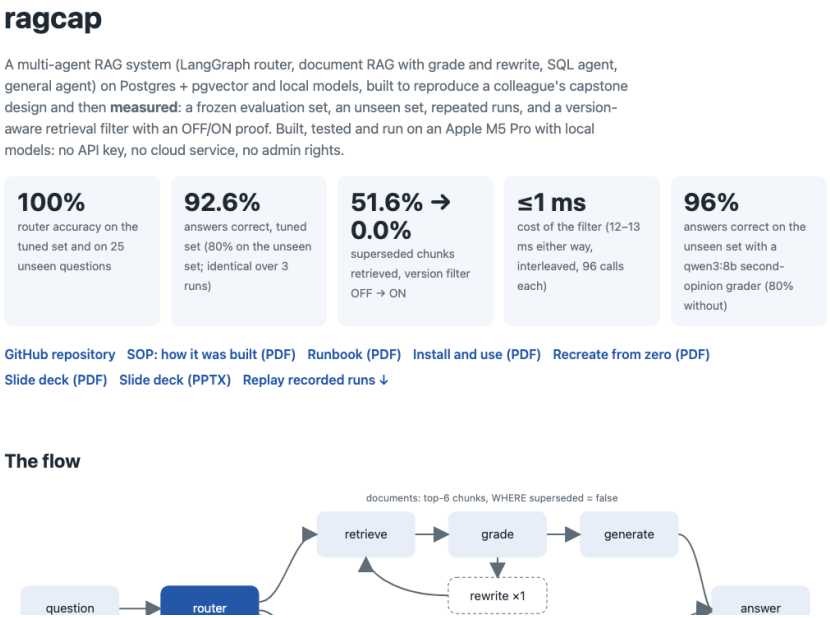
P&ID branch: drawings into the same index



qwen2.5vl:3b (5–12 s per drawing)	
Tags read correctly (3 drawings)	38/39, none invented
Right drawing retrieved	12/12
Answers correct, 12 frozen questions	4/12
what an instrument measures, on what	3/3
valve isolating a pump	1/3
controller + control valve	0/3
which drawing / unit	0/3

Synthetic drawings from one template with every tag in a ground-truth file (real P&IDs are confidential). The right drawing is always retrieved and a 7b vision model reads all 39 tags, but questions about connections (which valve is on the suction side) fail even with bigger models: the next step is storing connections as table rows.

See it, test it



Documents: SOP · Runbook · Recreate guide · Install guide

Live showcase (GitHub Pages, static): https://shreenathacc22.github.io/ragcap/

Results, the flow, screenshots, the documents, and a replay of 64 recorded runs: pick a question, see its route, node timings, answer, and for version cases the documents retrieved OFF vs ON.

Test it live (Ollama + Python 3.12; a few minutes):

```
git clone https://github.com/shreenathacc22/ragcap
cd ragcap && ./recreate.sh
```

Then try the scenario catalogue in docs/RECREATE.pdf, for example:

python scenarios.py → 12 pass, 5 known weaknesses, 0 unexpected

python b1_eval.py → old versions retrieved 51.6% OFF, 0% ON

"Which brokers are above 85% disk usage?" → SQL agent, brokers 3 and 5

Deploying the site to Vercel, step by step

1

Sign in

vercel.com → Continue with GitHub (the account that owns shreenathacc22/ragcap).

2

Import

Add New → Project → Import Git Repository → shreenathacc22/ragcap. Authorise the Vercel GitHub app for this repo if asked.

3

Settings

Framework Preset: Other. Root Directory: leave as the repo root. Build Command: empty. Output Directory: site (vercel.json already says so).

4

Deploy

Click Deploy; the first URL looks like ragcap-<hash>.vercel.app and opens the same site as GitHub Pages.

5

Domain

Settings → Domains → add ragcap.vercel.app (or any free name); paste the URL into README.md and the site, run publish_site.sh.

6

Keep it current

Every git push to main redeploys; the Deployments tab shows each build. CLI alternative: npm i -g vercel && vercel login && vercel --prod.

Why only the site: the pipeline needs Postgres + pgvector and Ollama, which neither Vercel nor Pages run. GitHub Pages is already live from the gh-pages branch (publish_site.sh); Vercel serves the identical site/ folder once imported.

Recreating it anywhere

```
git clone https://github.com/shreenathacc22/ragcap && cd ragcap
ollama pull llama3.2:3b; ollama pull nomic-embed-text   # no Docker needed
./recreate.sh          # DB, ingest, checks, fingerprints, OFF/ON ablation, smoke questions
.venv/bin/uvicorn api:app --port 8000     # http://127.0.0.1:8000/
./services.sh start && ./services.sh check   # after a reboot: CLEAN START OK
.venv/bin/python pid_eval.py --ingest   # P&ID branch (ollama pull qwen2.5vl:3b first)
```

Fingerprint	Value a correct rebuild must match
Database after ingest	16 docs, 16 chunks, 8 superseded, 1 table
Data checks	PASS; pair similarity 0.963–0.990; three negative tests reported
Retrieval ablation	OFF 99/192 (51.6%) → ON 0/192 (0.0%); trap 2/12 → 12/12; control 20/20
Golden sets (sha1 prefixes)	golden 68ec09bd…, held-out 02ddcfd6…, P&ID 10fe1aa6…

What this shows

A design can be reproduced on a laptop with no API key. What makes it credible is not the architecture but the measurement: a frozen test set, an unseen set, ranges over repeated runs, a filter proven OFF against ON, and negatives written down with their evidence.

On an Apple M5 Pro: lean models (~2.7 GB) answer 92.6% / 80%; a second-opinion grader lifts unseen questions to 96%. Next: P&ID connections as data, multi-hop, then deployment.

https://github.com/shreenathacc22/ragcap · https://shreenathacc22.github.io/ragcap/